

UX Research Data Modeling II: Task, Social, and Hybrid Models

Highlights

- Task structure model: Hierarchical task inventory
- Task sequence models
- Social model
- Hybrid models

9.1 INTRODUCTION

As we said in the previous chapter, these models are abstractions synthesized to represent UX research data from a particular perspective and used to inform the UX design process. Modeling should begin very early in the UX research process, starting with what you know at the beginning. Much of what you learn in the data-gathering and data-analysis sub-activities can go directly into creating and updating these models.

9.1.1 You Are Here

We begin each process chapter with a “You are here” picture of the chapter topic in the context of The Wheel, the overall UX design lifecycle template (Fig. 9.1). Within the understand needs UX design lifecycle activity, this chapter is the second of two about data modeling in which you use simple models to represent UX research data. This chapter covers task, social, and hybrid models.

9.1.2 Starting Point

By now you should have some of the models already started as you first encountered the UX research data. Your goal in this chapter is to continue synthesizing raw UX research data into sharable UX research data models.

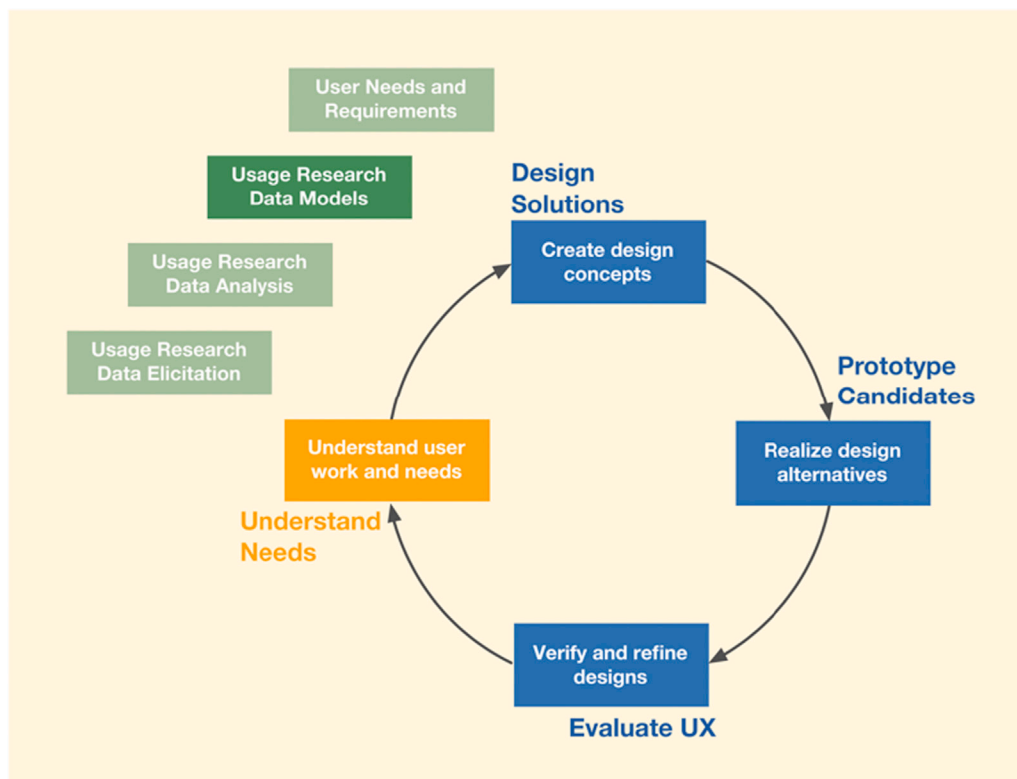


Fig. 9.1

You are here, in the second part of modeling, where we construct UX research data models within the understand needs activity, in the context of the overall Wheel UX lifecycle.

9.2 TASK STRUCTURE MODELS—HIERARCHICAL TASK INVENTORY

The primary task structure model is the hierarchical task inventory (HTI), a catalog of what users do or can do. If your product or system has a large number of user tasks in many different categories for many different user work¹ roles, an HTI is a good way to organize by compiling tasks and sub-tasks that must be supported for each user work role in the system design.

An HTI can be used:

- To organize and manage tasks and sub-tasks
- To organize user stories (Section 10.2.4) for the same

¹The term work broadly signifies anything people/users do while using products or systems (see Section 6.2).

- As a guide for creating a complete set of user stories as requirements
- To serve as a checklist for completeness in the emerging design (Constantine & Lockwood, 1999, p. 99)

9.2.1 Tasks Versus Functions

Informally, we may use the terms task and function more or less interchangeably when talking about the features of a system. When we wish to avoid confusion, however, we use task to refer to things a user does and function for things the system does.

When the point of view is uncertain, we sometimes see a reference to both. For example, if we talk about a situation where information is displayed/viewed, the two terms represent two viewpoints of the same phenomenon. It is clear that display is something the system does and that view is something the user does, as the user and system collaborate to perform the task/function.

9.2.2 Create an HTI Model

A simple task structure can be easily represented as a hierarchical (indented) list of tasks and sub-tasks. More complex task structures are best represented in an HTI diagram, where large, more general tasks are decomposed into smaller, more specific, and more detailed tasks. Hierarchical task inventories can be constructed top-down and/or bottom-up.

9.2.3 Hierarchical relationships

Location within the depth of the diagram indicates a hierarchical relationship between tasks and sub-tasks. If task A is immediately above task B in an HTI diagram (Fig. 9.2), then it means that task B is a sub-task of task A, and task A

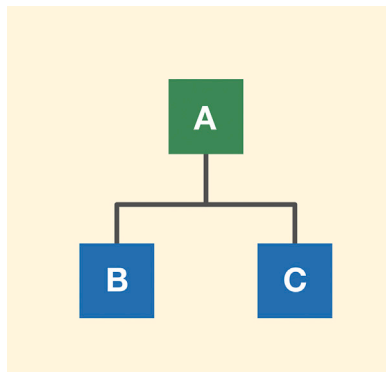


Fig. 9.2
Hierarchical relationship of task A, the supertask, and tasks B and C, sub-tasks.

is a supertask of task B. The litmus test for this relationship is: If you are doing task B, then you are necessarily also doing task A because task B is part of task A.

The best way to name tasks in this structure is as an action-object pair, such as “add appointment” or “configure parameters,” or in a verb-adjective-noun triple, such as “configure control parameters.”

9.2.4 Avoid temporal implications

The hierarchical relationship does not show temporal ordering. In Fig. 9.3, we depict an incorrect attempt at a hierarchical relationship because selecting a gear is not part of starting the engine—that is, it fails the litmus test mentioned earlier.

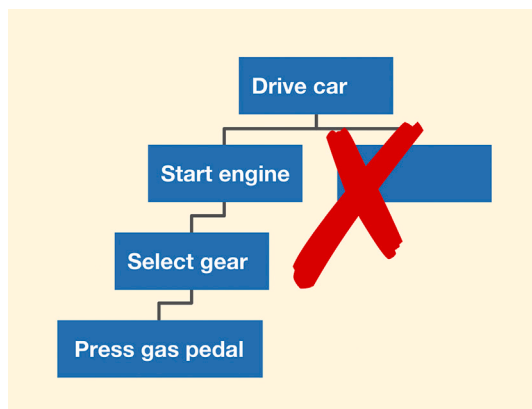


Fig. 9.3

An incorrect hierarchical relationship attempting to show temporal sequencing.

Example: A First-Level HTI Diagram for MUTTS

Starting at the very highest level of tasks for MUTTS, you have the major task sets performed by each of the work roles, such as the financial administrator, the database administrator, the ticket seller, the event manager, and the advertising manager. Using an action-object approach to task naming, these major task sets may be called manage finances, manage database, and so on (Fig. 9.4).

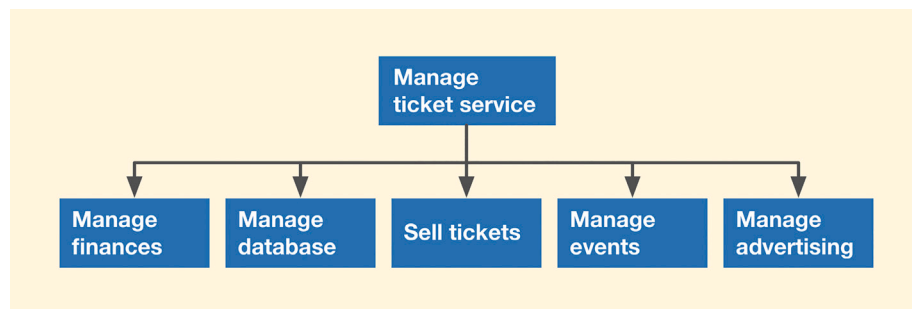


Fig. 9.4

Sketch of the top levels of a possible hierarchical task inventory diagram for MUTTS.

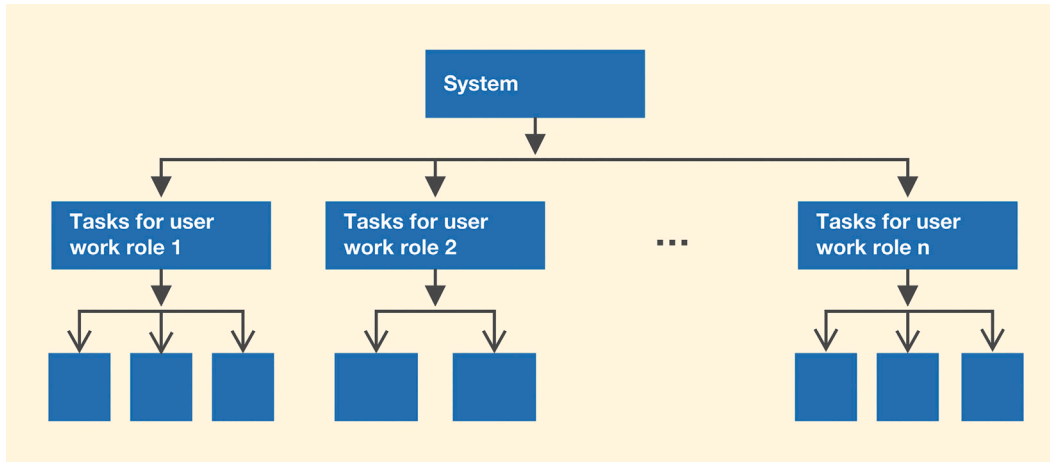


Fig. 9.5

Separation by user work roles at the top of a task hierarchy.

9.2.5 HTI can often be decomposed by user work role

The full HTI for a non-trivial system can be enormous. Fortunately, there is a way to control this complexity. Because the set of tasks performed by one user work role is usually separate from those performed by others, you can usually partition the full HTI by user work roles, resulting essentially in a separate HTI diagram for each user work role (Fig. 9.5).

The team can usually consider each user work role diagram more or less separately in analysis and design. Of course, there are system-spanning issues and parts of the design that involve multiple user work roles, but this imperfect compartmentalization helps a great deal by allowing UX designers to look at the whole system one piece at a time.

The sketch of an HTI in Fig. 9.6 is only for the ticket seller role in MUTTS. As stated earlier, the HTI of Fig. 9.6 can be used as a checklist to be sure you create interaction designs for all the tasks.

Exercise 9.1 HTI for Your Product or System

Goal: Practice creating an HTI diagram.

Activities: Using your task-related work activity notes and what you know about your product or system, make a simple HTI diagram for your system.

Deliverables: Simple HTI diagram(s) for the product or system of your choice.

Schedule: About 1 hour should be enough to get what you need from this exercise.

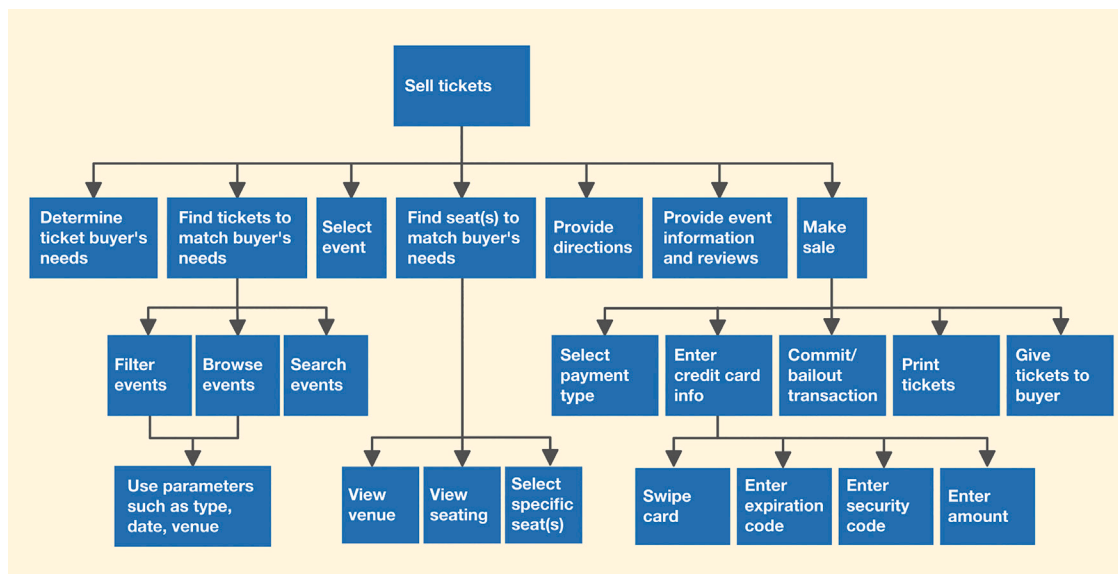


Fig. 9.6

Example HTI for the ticket seller role in MUTTS.

9.3 TASK SEQUENCE MODELS

Task sequence models are step-by-step descriptions of how users perform tasks with a product or system. Task sequence information about steps within a task and navigation through a task comes from various sources in data gathering. Only some of it will come from users. Task sequence modeling may begin with models for important workflows, building up details via targeted UX research.

Task sequence models can be thought of as embryonic interaction designs. User actions and system responses are often separated into dual (or multiple) swim lanes. Task descriptions have certain boilerplate parameters, as described later. Some sample work activity notes suitable as inputs to a task sequence model are:

I need to see a confirmation page showing exactly what I'm buying before I pay.
As part of the ticket selection process, I need to see the layout of seats in the venue.
The kiosk should remind people to take the ticket and credit card at the end of the transaction.

9.3.1 Scenarios as a Simple Task Sequence Model

A scenario is a story about specific people performing work activities in a specific work situation within a specific work context, told in a concrete narrative style as if it were a transcript of a real usage occurrence (Carroll,

1995). Scenarios are deliberately informal, open-ended, economical, and fragmentary narrative depictions of key usage situations happening over time.

A usage scenario is a description of a way someone uses an existing product or system. A design scenario is a description of envisioned usage of a product or system being designed.

Example: Usage Scenario for a Menu Planning Application

This example is taken, with permission, from a graduate-level HCI class student project. Written at first at a high level, a substantial set of these scenarios was expanded to include more detail and to check the emerging design against requirements, identify system states, and identify tasks and even early prototype interaction features.

Bob likes to cook so many different types of food that his kitchen is cluttered with cookbooks. In the past, locating recipes that he is “in the mood for” was a lot of work. Now, however, he has Menu-Bot to help. He is planning a dinner party for six guests and has already made up his mind to serve baked salmon with an almond sauce on wild rice, along with lemon butter squash. So he enters just those dishes directly into the system and gets some good recipes that will go well together. However, he still needs an appetizer and dinner wine, so he lets Menu-Bot recommend them and, seeing that they are good, he adds them to the dinner menu.

Exercise 9.2 Usage Scenarios as Simple Task Sequence Models for Your Product or System

Goal: Practice writing usage scenarios as early, simple task-sequence models.

Activities: Select one or two good representative task threads for one of your user work roles (e.g., the customer).

- Write a couple of detailed usage scenarios, referring to user roles, tasks, actions, objects, and work context.
- Work quickly; you can clean it up as you go.

Deliverables: A few usage scenarios to share and discuss.

Schedule: About 1 hour should be enough time for this one.

9.3.2 Step-by-Step Task Sequence Models

9.3.2.1 Task and step goals

Called the user “intent” by Beyer and Holtzblatt (1997), a task or step goal is the purpose or reason for doing the task or taking the step. It is a description of what the user wants to accomplish by doing the task or step.

9.3.2.2 Task triggers

A task trigger (Beyer & Holtzblatt, 1997) or step trigger is something that happens, an event or activation condition, that leads that user to initiate a given task or task step. Triggers are easy to identify in your data notes in UX research analysis. For example, an incoming customer phone call might lead to filling out an order form.

Other examples of task triggers include the following: New work arrives in the user's inbox, a new aircraft appears on the air traffic controller's screen, an email request arrives, and the calendar says a report is due soon.

9.3.2.3 Task barriers

Indicators of task barriers in your UX research data include user problems and errors that get in the way of successful, easy, and satisfying task or step completion. Task barriers are pain points or choke points that frustrate users and block productivity or flow. To indicate barriers in our data models, we use the Beyer and Holtzblatt symbol of a red lightning bolt (⚡), which you should put at the beginning of an indented line explaining the barrier. If the user's reaction or response to a barrier is known through the UX research data, then add a brief description of that directly after the barrier description among the task steps.

Task barriers are of special interest in UX design because they point out where users have difficulties in the work practice, which in turn are key opportunities for improvement in the design.

9.3.2.4 Information and Other Needs in Tasks

An important component of a task description is the identification of unmet user information and other needs at any step, one of the largest sources of barriers in task performance. The UX research data gathering and analysis processes and modeling can help you identify these needs, which you can represent with specific annotations. Just before the step in which the need occurs, add an indented line beginning with a red block **N**, followed by a description of the need.

9.3.2.5 Begin With a Linear List of Basic Steps

- In user stories or here in modeling, as you encounter work activity notes that mention tasks, task steps, or sub-tasks, merge them into the appropriate evolving task sequence model.

- Sequential steps can be written as an ordered list without the need for flowchart-style arrows to show the flow:
 - Linear lines of text are less cluttered and easier to read.
- Start by showing the most common steps users would take:
 - This is sometimes called the “happy path” or “go path.”
 - This gives a quick and easy-to-understand idea of the basic task without clutter of special cases.

Going from a set of related user stories (see [Section 10.2](#)) to a related task sequence model, to a wireframe design (see [Section 16.6](#)) is a very natural path that works well in agile UX design. This is another example of how modeling can be distributed throughout the UX lifecycle.

Example: Rudimentary Task Sequence Model for MUTTS

Consider an extremely simple step-by-step task sequence representation for a MUTTS ticket transaction:

MUTTS Ticket Buyer

1. Wait in line until turn.
3. Describe event for which tickets are wanted.
5. State number of tickets needed.
7. Select seats.
9. Pay with credit card.

MUTTS Ticket Seller

2. Greet ticket buyer.
4. Look up event in database. How many tickets will be needed?
6. Look up venue seating chart. Describe available options by seat location and price.
8. Calculate and state total cost. How would you like to pay?
10. Give tickets, receipt, and card to ticket buyer.

You can see that this is a bare task sequence skeleton. You will add other steps and details as you learn about them.

Exercise 9.3 More Detailed Step-by-Step Task Sequence Model for MUTTS

Task goal: Help a ticket buyer choose and buy a ticket for entertainment for this Friday night (performed by ticket seller on behalf of ticket buyer).

Task trigger: Ticket buyer arrives at the MUTTS ticket window on the way home from work on a Thursday evening, thinking ahead to the weekend.

Note: It can help analysis and discussion to number the steps so you can refer to them, as we do in this example.

Ticket Buyer	Ticket Seller
1. Tell ticket seller about general goal of wanting to find an entertainment event for the next night (Friday night).	
2. Ask agent about available types of entertainment.	3. Tell ticket buyer that there are plays, concerts, movies, and sports.
4. Not enough information yet to decide on the category. Ask to see examples of different types.	

Step goal: Consider examples of entertainment events.

5. Ask what events are available for Friday night.
--

Barrier ⚡: Agent sees that the number of results is too large to sort through or tell the customer about.

Response to barrier:

	6. Ask customer how to filter results or narrow it down (e.g., “Tell me more about what you like”).
7. Ask about something within reasonable walking distance downtown or near a Middleburg bus stop.	8. Tell about some possibilities.

Task continues:

9. Think about the list of possibilities.

⚡: It is difficult to think about specific events while remembering all the others given orally on the list.

Trigger: Movies seem attractive to ticket buyer.

Goal: Find a movie to see.

- | | |
|---|-------------------------------|
| 10. Tell agent about switching focus to just movies. | |
| 11. Tell agent to use the same criterion about being within reasonable walking distance downtown or near a Middleburg bus stop. | 12. Tell about possibilities. |
| 13. Consider possibilities and find a few candidates. | |

Trigger for interruption to embedded task: Thinks a friend might also like these movies.

N: Needs to know friend's opinion of the selections.

Goal: Contact a friend to help narrow these last few choices down and pick something together.

- | |
|--------------------------------|
| 14. Ask agent to please wait. |
| 15. Call friend on cell phone. |
| 16. Make choice with friend. |

Trigger: Choice made, ready to buy two tickets.

Goal: To buy tickets.

- | | |
|--|--|
| 17. Request to buy two tickets for selected movie. | 18. Set up transaction in computer. |
| 20. Give agent credit card. | 19. Ask: Cash or credit card? |
| 22. Sign credit transaction. | 21. Swipe card. |
| | 23. Print tickets and receipt. |
| | 24. Give printed tickets and return credit card and receipt. |

9.3.3 Beyond Linear Task Sequence Models

At the beginning, individual task sequence models will be mostly linear paths. Later you can add the most important branching or looping (see later) to cover conditional and iterative cases.

Branching and looping. Although step-by-step task sequence models are primarily for capturing linear sequences of representative task steps,

sometimes you encounter a point in the work practice where there is a choice. For example, an initial task sequence model for an online purchase may not show a decision point where the user can pay with either a credit card or PayPal. It would just be a set of steps for the task of buying a ticket with, say, a credit card. Later, a separate path for the alternative of paying with PayPal is merged, introducing a decision-making point and branching, represented as arrows on the left-hand side of [Fig. 9.7](#).

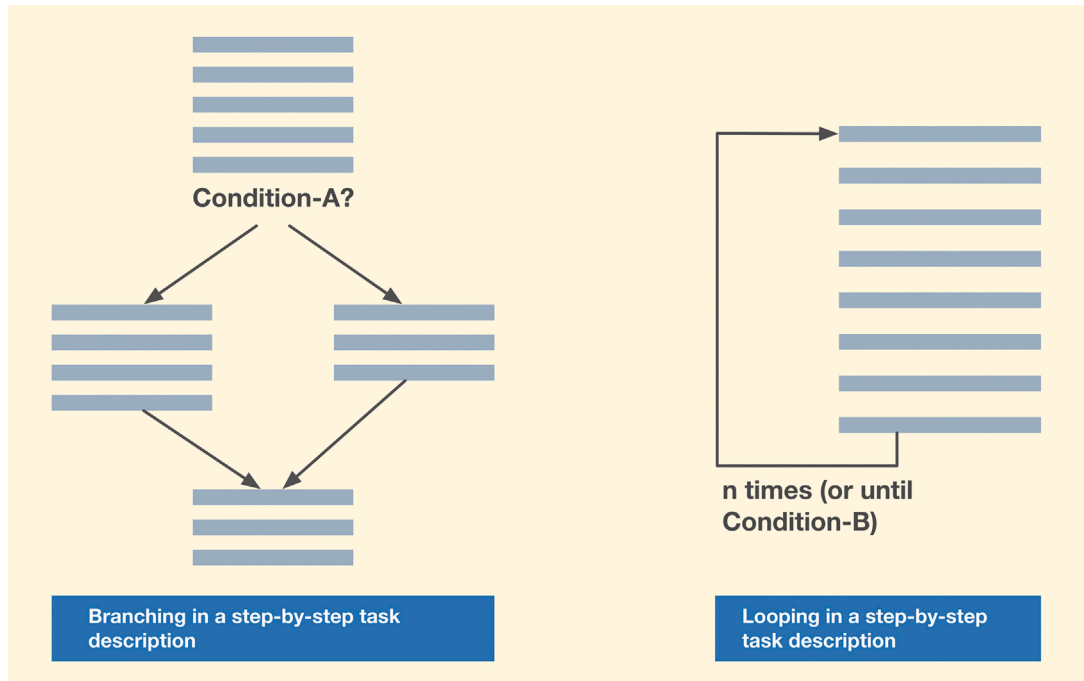


Fig. 9.7

Branching and looping structures within step-by-step task sequence models.

Similarly, if you observe iteration of a set of tasks or task steps, then you can represent that as shown on the right-hand side of [Fig. 9.7](#). For sets of steps that are repeated or iterated, note the number of iterations or the condition for termination.

Example: Task Sequence Branching and Looping for MUTTS

In [Fig. 9.8](#), we show a sketch of task sequence representation for selling tickets with MUTTS. Note several instances of looping to iterate parts of the task and, in the bottom box, branching to accommodate two different cases.

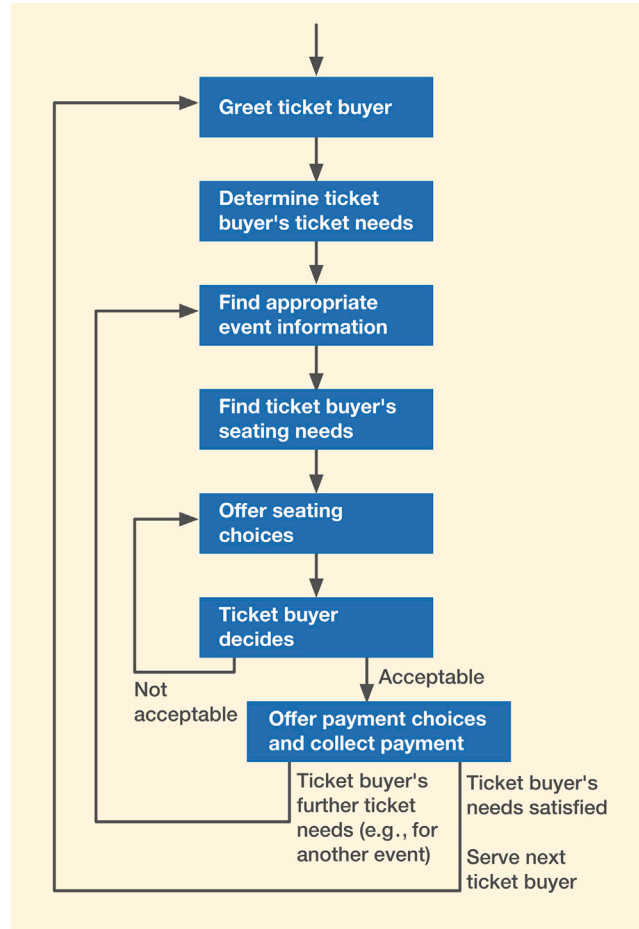


Fig. 9.8
Task sequence branching
and looping for MUTTS.

9.3.4 Essential Use Case Task Sequence Models

By combining the best characteristics of step-by-step task descriptions and software use cases, Constantine and Lockwood (1999, p. 100) created essential use cases as an alternative task sequence modeling technique. A use case is a software engineering artifact for documenting functional requirements, especially for object-oriented development, of a system. In answer to the need for something more effective than use cases in identifying interaction design requirements, Constantine (1994a; 1995)

created the variation called “essential use cases.” An essential use case (Constantine & Lockwood, 2003):

- Is a structured narrative
- Is expressed in the language of users in the work domain
- Describes a task associated with a single user intention or goal
- Is abstract

9.3.4.1 Naming Essential Use Cases

According to Constantine and Lockwood (2003), each essential use case is named with a continuing verb to indicate an ongoing intention, plus a fully qualified object (e.g., “Buying a movie ticket”).

9.3.4.2 An Essential Use Case Is Abstract

The term essential refers to the fact that the description is abstract (extraneous details removed) in the sense that an essential use case:

- Contains only steps that are the essence of the task
- Represents only one possible task thread, usually the simplest thread without alternatives or special cases
- Is expressed as a pure work-domain representation, independent of technology or how it looks in the UX design
- Is a skeleton on which a task description can be woven
- Captures what users intend to do and why but not how in terms of user actions, such as clicking a button

Essential use cases help structure the interaction design around core tasks. These are efficient representations, getting at the essence of what the user wants to do and the corresponding part played by the system. To illustrate, in Constantine and Lockwood’s (2003) ATM example, the user’s first step is expressed as an abstract purpose, the “what” of the interaction: “identify self.” They do not express it in terms of a “how”; for example, they do not say the first step is to “insert bank card.” This is a deceptively simple example of a very important distinction.

Example: Essential Use Case for Ticket Kiosk System

Table 9.1 contains an example, cast in the same fashion as Constantine and Lockwood (2003). Notice how these descriptions are abstract.

TABLE 9.1

Example Essential Use Case: Paying for a Ticket Purchase Transaction (With a Credit or Debit Card)

Ticket Buyer Intention	Ticket Seller and System Responsibility
1. Express intention to make a transaction.	2. Request user to identify self.
3. Identify self.	4. Request to state desired transaction.
5. State desired transaction.	6. Initiate possible negotiation of transaction parameters.
7. Participate in possible transaction negotiations.	8. Summarize transaction and cost; request transaction confirmation.
9. Submit confirmation to make payment.	10. Conclude transaction.

Note how the abstraction leaves room for design. For example, user identification could be accomplished via a credit card. Transaction confirmation could be submitted in the form of a signature, and the transaction could be concluded with a receipt.

Exercise 9.4 Task Sequence Model for Your Product or System

Goal: Create a somewhat detailed essential use case model for a key task in your product or system.

Activities: Select your key task and give it a name.

Break one possible performance thread for this task into steps and where appropriate for each step:

- Identify user intent.
- Indicate task triggers.
- Note possible breakdown points.

Deliverables: A task sequence, written out as an essential use case model.

Schedule: Should take about 30 minutes.

9.3.5 User Workflow Diagrams: The Next Step in Representing Task Sequencing and Navigation

A user workflow diagram (see [Section 18.3.2](#)) is a simple task sequence model used to highlight user workflow when the interaction involves multiple choices and navigation paths. A user workflow diagram is a directed graph in which nodes are states that correspond to screens (i.e., user interaction states), and arcs (or arrows) are transitions between states resulting from user actions or system events. User workflow diagrams lead directly to wireflow prototypes, showing navigational flow among nodes of wireframe prototypes.

User workflow diagrams can also help understand the lifecycle of an information object in the work practice ecology. For example, the user workflow diagram of an event object includes created (by event manager in collaboration with event sponsor), updated, accessed (by multiple roles in multiple contexts), associated with venue and ticket objects, and archived after a period of time. The transitions among the various states and the triggers that cause such transitions are shown using arcs with annotations.

Although a user workflow diagram can be used at any level of detail, initial user workflow diagrams are most useful for establishing initial wireframes if you stick with the main navigational paths and leave out peripheral detail, such as error checking or confirmation dialogue. In a transactional system, the flow model can get very complex, and edge cases can multiply. An abstract user workflow diagram can help you find the essence of the flow to form the backbone of the simplest version of the wireframe design. When you get to designs of wireframes for system screens, you can think of each screen as being a state in the user workflow diagram. You are designing for things that live in that state. To avoid repetition with the full description of user workflow diagrams, we refer you to [Section 18.3.2](#) for more about what user workflow diagrams are and how to make and use them.

9.4 THE SOCIAL MODEL

The social model captures the culture of the shared workplace, communal issues of the users' organizational workplace as a social setting, and how they impact the way things get done in the work environment. Our social model is based on Beyer and Holtzblatt's (1997) cultural model.

In practice, the social model is the least commonly used model, especially in agile environments. It is needed only when social and cultural interactions among the people involved in the work practice are complex and/or problematic.

9.4.1 Simplified Approach to the Social Model

Formally, a social model has been represented as a kind of bubble diagram with influencers within the culture represented as bubbles, and cultural or social relationships and influences represented as arrows between bubbles. See the cultural model of Beyer and Holtzblatt (1997).

In practice, we have found that the social model can be:

- Difficult to make because the information needed is seldom volunteered by users during data gathering
- Difficult to understand because the arcs and labels of a graphic representation can easily become too busy and too detailed

- Overly complicated because all that structure is not often necessary
- Difficult to apply to inform UX design because technical design solutions for social problems can be elusive

We include this model mainly for completeness. For those who want or need to make a social model, we have tried to reduce it to the most important aspects represented within a structured textual representation. See the example of a social model for MUTTS, later.

9.4.2 Identify Active Entities

Active entities in a social model include all user work roles and can include any non-user agent or force that participates in, influences, or is impacted by the work practice.

In addition to the work roles and entities internal to the client organization, there are external roles that interact with work roles, including outside vendors, customers, regulatory agencies, the government, the market, or the competition.

Example: List of Social Model Entities for MUTTS

Our example of a social model for MUTTS begins with the roles. We identify the ticket seller and ticket buyer as the main roles, represented as list items at the highest level. You will almost always want to include the ambience and work domain as a non-human entity. The administrative supervisor, database administrator, and office manager are also shown in this list for Ticket Kiosk System:

- Ticket seller
- Ticket buyer
- Ambience
- Work domain
- Administrative supervisor
- Database administrator
- Office manager

9.4.3 Identify Types of Issues, Pressures, Worries, and Concerns

Categories of social model issues include matters such as:

- Concerns and worries
- Influences exerted or felt
- Job pressures
- Issues affecting job performance

More specific examples include:

- The overall flavor or feeling in the workplace
- Organizational philosophy and culture
- Workplace ambience and environmental factors (e.g., toxic work environment)
- Professional and personal goals of workers
- Political structure and realities
- Thought processes, mindsets, policies, feelings, attitudes, and terminology that occur in the work environment
- Legal requirements and regulations
- Organizational policies
- Subversive activities, destructive reactions to dissatisfaction, frustration
- Conflicts, inter-role frictions, and animosities
- Power influences, employees pulling rank based on official job titles
- Leadership becomes bullying

Example: A Social Model for MUTTS

Here is an excerpt of what a social model might look like for MUTTS:

- Ticket buyer
 - Feels pressure:
 - From those in line to get done fast
 - To get good seats for popular events
 - Worries about:
 - The system completing the transactions correctly
 - Losing the well-liked personal service if it is changed to a kiosk
 - Losing tickets or money if the system has an error
 - Losing money due to the no-refund policy
 - Experiences barriers to work in the form of:
 - Noise and distraction of public location interfere with thinking and decision making
 - Cannot see all the choices upfront
- Ticket seller
 - Feels pressure:
 - To please customers
 - Peak periods mean fast pace and hard work
 - Pushback from users about the no-refund policy
 - Worries about:
 - Complaints about service that can lead to poor job reviews

- Is subject to external influences of:
 - The administrative supervisor, who shows up occasionally and causes stress
 - The administrative supervisor, who is not familiar with daily operations and can impose unrealistic expectations

Exercise 9.5 A Social Model for Your Product or System

Goal: Practice making a social model diagram.

Activities: Identify active entities, such as work roles, and represent as bullets in a list. Include:

- Groups and sub-groups of roles and external roles that interact with work roles
- System-related roles, such as a central database
- Workplace ambience and its pressures and influences

Next identify:

- Concerns and perspectives to be represented as sub-bullets in a list
- Social relationships, such as influences between entities, to be represented as sub-bulleted items under appropriate entities
- Barriers, or potential barriers, in relationships between entities to be represented as red bolts of lightning (⚡).

Deliverables: One social model diagram for your system with as much detail as feasible.

Schedule: This could take a couple of hours.

9.5 HYBRID MODELS

Although we describe each UX research data model separately for clarity, in practice models are often combined for efficiency. For example, because the flow model includes at least the major user work roles, you usually do not need a separate user work role model. You can just label the user work role nodes of the flow model with the user work role names and annotate them with user class characteristics. Later, you can even label flow model nodes with corresponding personas.

The goal of modeling is to synthesize a useful perspective on the work domain. Purity of each model is not the goal. Do what it takes to capture what you learn about the work domain. For example, if the work practice centers around a physical space, such as a ticket office, combine the physical model and flow model into a hybrid model.

9.6 TRADE-OFFS BETWEEN THE WORK ACTIVITY AFFINITY DIAGRAM AND THE MODELS

Building the work activity affinity diagram (WAAD; see [Section 7.5](#)) is an important step toward understanding UX design needs. In [Chapter 8](#) and this chapter, we have learned about building the UX research data models. The WAAD and data models will often overlap, representing different perspectives of the data gathered. The point is to use our guidance on doing only what is necessary, meaning in some small projects in a domain simple interaction, simple quadrant of the system complexity space (see [Section 3.2.1.4](#)), a WAAD might be all you do or need. For most other cases, key models are necessary, such as the flow model and perhaps a WAAD.

If the project context requires more modeling to understand the work practice, there is probably less need for a WAAD because most work activity notes would have been used as inputs to models. If the context is that the UX team is new to the work practice, perhaps they need both WAAD and a bunch of models.

9.7 EFFICIENT DATA HANDLING FOR PROFESSIONAL UX PRACTITIONERS

As you get experience with the UX research activities, you can save the time and effort of intermediate steps by integrating the raw UX research data, still in the head of the practitioner, directly into the corresponding representation, such as work activity notes, a WAAD (see [Section 7.5](#)), data models (see [Chapters 8](#) and this chapter), or user stories and requirements (see [Chapter 10](#)). For efficient data modeling, incorporate observations made in data gathering directly into the evolving data models, skipping over writing raw data notes and distilling into work activity notes entirely.

Example: Work Activity Notes Represented Directly in the Flow Model

As an example, this work activity note can be represented as a barrier in the workflow of the ticket buyer:

It is usually difficult to get enough information about events from a ticket seller at the ticket window.

Here are some other work activity notes from the Ticket Kiosk System UX research analysis that can be directly represented in a flow model.

It will be very important to have a confirmation page showing exactly what I am buying before I pay (as good online sites do).

The kiosk should remind people where the ticket comes out.

Part of the ticket selection process is the option to view the map to the venue.

The kiosk should remind people to take the ticket and credit card (at the end of the transaction).

9.8 WRAP-UP

9.8.1 Barrier Summaries Across All Models

Barriers to usage are of special interest because they point out where users have difficulties in the work practice. Anything that impedes user activities, interrupts workflow or communications, or interferes with the performance of work responsibilities is a barrier to the work practice. Because these barriers also represent key opportunities for improvement in the design, it is useful to collect them all into a single summary.

9.8.2 Post Data Models in Your UX Design Studio

Make your models visual in sketches and posters prominently displayed as part of the immersion in your design studio. Assemble your artifacts into an organized display.

Considerations about posting in your UX design studio.

- In early stages, it may be best to sketch your diagrams and figures on whiteboards, where it is easy to discuss and update them.
- Later, you can transfer the diagrams to nicely printed posters taped to the wall.
- Assemble your artifacts into an organized collection on a central worktable.
- At some point in projects of any realistic size, you will end up with more materials to post than you have space for posting, so:
 - Be judicious; trim out the less important items.
 - Post artifacts and diagrams that will have importance throughout the entire project.
 - Diagrams, images, and sketches that you will use less frequently could be kept in your laptop and, when needed, shown via a projector.
 - Beyond this, larger amounts of shared information can be kept in sharable files (e.g., in Dropbox or Google Drive).

9.9 SEGUE

In this and the previous chapter we traversed the work-domain space through different models, each illuminating a specific dimension of the work practice. In the next chapter, we get into identifying requirements and capabilities needed in the envisioned system.